OS Practical Answer

Assignment 1 (1 gcc filename.c -o filename 2 ./filename)

Q1:

Add the following functionalities in your program

a) Accept Available
b) Display Allocation, Max
c) Display the contents of need matrix
d) Display Available

Process	Allocation			MAX		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

```
#include <stdio.h>
#define P 5   // Number of processes
#define R 3   // Number of resources
int main() {
   int allocation[P][R] = {
      {0, 1, 0},
      {2, 0, 0},
      {3, 0, 2},
      {2, 1, 1},
      {0, 0, 2}
   };
   int max[P][R] = {
      {7, 5, 3},
      {3, 2, 2},
      {9, 0, 2},
      {2, 2, 2},
      {4, 3, 3}
   };
   int available[R];
   int need[P][R];
   // Accept Available Resources
   printf("Enter Available Resources (A B C): ");
   for (int i = 0; i < R; i++)
      scanf("%d", &available[i]);
   // Calculate Need Matrix
   for (int i = 0; i < P; i++)
      for (int j = 0; j < R; j++)
```

```
            need[i][j] = max[i][j] - allocation[i][j];
    // Display Allocation Matrix
    printf("\nAllocation Matrix:\n");
    printf("    A B C\n");
    for (int i = 0; i < P; i++) {
        printf("P%d  ", i);
        for (int j = 0; j < R; j++)
            printf("%d ", allocation[i][j]);
        printf("\n");
    }
    // Display Max Matrix
    printf("\nMax Matrix:\n");
    printf("    A B C\n");
    for (int i = 0; i < P; i++) {
        printf("P%d  ", i);
        for (int j = 0; j < R; j++)
            printf("%d ", max[i][j]);
        printf("\n");
    }
    // Display Need Matrix
    printf("\nNeed Matrix:\n");
    printf("    A B C\n");
    for (int i = 0; i < P; i++) {
        printf("P%d  ", i);
        for (int j = 0; j < R; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    // Display Available Resources
    printf("\nAvailable Resources:\n");
    printf("A B C\n");
    for (int i = 0; i < R; i++)
        printf("%d ", available[i]);
    printf("\n");
    return 0;
}
```

Q2:Modify above program so as to include the following: a) Accept Request for a process b) Resource request algorithm c) Safety algorithm Consider a system with ‘n’ processes and ‘m’ resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not.

```
#include <stdio.h>
#include <stdbool.h>
#define P 5   // processes
#define R 3   // resources
int allocation[P][R] = {
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1},
    {0, 0, 2}
};
int max[P][R] = {
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2},
    {4, 3, 3}
};
int available[R] = {3, 3, 2};
int need[P][R];
void calculateNeed() {
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max[i][j] - allocation[i][j];
}
bool safetyAlgorithm() {
    int work[R];
    bool finish[P] = {false};
    int safeSeq[P];
    int count = 0;
    for (int i = 0; i < R; i++)
        work[i] = available[i];
    while (count < P) {
        bool found = false;
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                int j;
                for (j = 0; j < R; j++)
                    if (need[i][j] > work[j])
                        break;
                if (j == R) {
                    for (int k = 0; k < R; k++)
```

```
                work[k] += allocation[i][k];
            safeSeq[count++] = i;
            finish[i] = true;
            found = true;
        }
    }
  }
  if (!found) {
    printf("\nSystem is NOT in safe state\n");
    return false;
  }
}
printf("\nSystem is in SAFE state\nSafe Sequence: ");
for (int i = 0; i < P; i++)
  printf("P%d ", safeSeq[i]);
printf("\n");
return true;
}
void resourceRequest(int pid, int request[]) {
  for (int i = 0; i < R; i++) {
    if (request[i] > need[pid][i]) {
      printf("\nRequest exceeds maximum need\n");
      return;
    }
    if (request[i] > available[i]) {
      printf("\nResources not available\n");
      return;
    }
  }
  for (int i = 0; i < R; i++) {
    available[i] -= request[i];
    allocation[pid][i] += request[i];
    need[pid][i] -= request[i];
  }
  if (safetyAlgorithm())
    printf("\nRequest can be GRANTED\n");
  else
    printf("\nRequest CANNOT be granted\n");
}
int main() {
  int pid;
  int request[R];
```

```
    calculateNeed();
    printf("Enter Process Number (0 to 4): ");
    scanf("%d", &pid);
    printf("Enter Request for resources (A B C): ");
    for (int i = 0; i < R; i++)
        scanf("%d", &request[i]);
    resourceRequest(pid, request);
    return 0;
}
```

Assignment 2

Q3: Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options as mentioned above and implement each option.

```
#include <stdio.h>

int main() {
    int n, disk[50], i, choice;
    int start, length, flag;
    printf("Enter number of disk blocks: ");
    scanf("%d", &n);
    // Initialize all blocks as free (0)
    for (i = 0; i < n; i++) {
        disk[i] = 0;
    }
    do {
        printf("\n--- Sequential File Allocation ---");
        printf("\n1. Allocate File");
        printf("\n2. Display Disk Status");
        printf("\n3. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1:
            printf("Enter starting block: ");
            scanf("%d", &start);
            printf("Enter length of file: ");
            scanf("%d", &length);
            flag = 0;
            // Check availability
            for (i = start; i < start + length; i++) {
                if (disk[i] == 1 || i >= n) {
```

```
                flag = 1;
                break;
            }
        }
        if (flag == 0) {
            for (i = start; i < start + length; i++) {
                disk[i] = 1;
            }
            printf("File allocated successfully.\n");
        } else {
            printf("File cannot be allocated.\n");
        }
        break;
    case 2:
        printf("\nDisk Block Status:\n");
        for (i = 0; i < n; i++) {
            printf("Block %d : %s\n", i, disk[i] ? "Allocated" : "Free");
        }
      break;
    case 3:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice.\n");
    }
  } while (choice != 3);
  return 0;
}
```

Q4: Write a program to simulate Linked file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options as mentioned above and implement each option.

```
#include <stdio.h>
int main() {
  int n, disk[50], next[50];
  int i, choice, start, length, curr;
  printf("Enter number of disk blocks: ");
  scanf("%d", &n);
  // Initialize disk blocks as free
  for (i = 0; i < n; i++) {
    disk[i] = 0;
    next[i] = -1;
```

```
}
do {
    printf("\n--- Linked File Allocation ---");
    printf("\n1. Allocate File");
    printf("\n2. Display Disk Status");
    printf("\n3. Exit");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);
    switch (choice) {
    case 1:
        printf("Enter starting block: ");
        scanf("%d", &start);
        printf("Enter number of blocks needed: ");
        scanf("%d", &length);
        if (disk[start] == 1) {
            printf("Starting block already allocated.\n");
            break;
        }
        curr = start;
        disk[curr] = 1;
        for (i = 1; i < length; i++) {
            int j;
            for (j = 0; j < n; j++) {
                if (disk[j] == 0) {
                    disk[j] = 1;
                    next[curr] = j;
                    curr = j;
                    break;
                }
            }
            if (j == n) {
                printf("Not enough free blocks.\n");
                break;
            }
        }
        next[curr] = -1;
        printf("File allocated successfully.\n");
        break;
    case 2:
        printf("\nBlock\tStatus\tNext\n");
        for (i = 0; i < n; i++) {
            printf("%d\t%s\t%d\n", i,
```

```
                disk[i] ? "Allocated" : "Free",
                next[i]);
            }
            break;
        case 3:
            printf("Exiting program.\n");
            break;
        default:
            printf("Invalid choice.\n");
        }
    } while (choice != 3);
    return 0;
}
```

Q5: Write a program to simulate Indexed file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options as mentioned above and implement each option.

```
#include <stdio.h>
int main() {
    int n, disk[50], indexBlock, fileBlocks[50];
    int i, choice, length;
    printf("Enter number of disk blocks: ");
    scanf("%d", &n);
    // Initialize disk blocks as free
    for (i = 0; i < n; i++) {
        disk[i] = 0;
    }
    do {
        printf("\n--- Indexed File Allocation ---");
        printf("\n1. Allocate File");
        printf("\n2. Display Disk Status");
        printf("\n3. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1:
            printf("Enter index block: ");
            scanf("%d", &indexBlock);
            if (disk[indexBlock] == 1) {
                printf("Index block already allocated.\n");
                break;
            }
```

```
        printf("Enter number of blocks needed for file: ");
        scanf("%d", &length);
        printf("Enter block numbers:\n");
        for (i = 0; i < length; i++) {
            scanf("%d", &fileBlocks[i]);
        }
        // Check availability
        int flag = 0;
        if (disk[indexBlock] == 1)
            flag = 1;
        for (i = 0; i < length; i++) {
            if (disk[fileBlocks[i]] == 1) {
                flag = 1;
                break;
            }
        }
        if (flag == 0) {
            disk[indexBlock] = 1;
            for (i = 0; i < length; i++) {
                disk[fileBlocks[i]] = 1;
            }
            printf("File allocated successfully.\n");
        } else {
            printf("File allocation failed.\n");
        }
        break;
    case 2:
        printf("\nDisk Block Status:\n");
        for (i = 0; i < n; i++) {
            printf("Block %d : %s\n", i,
                   disk[i] ? "Allocated" : "Free");
        }
        break;
    case 3:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice.\n");
    }
  } while (choice != 3);
  return 0;
}
```

Assignment 3

Q6: Write an OS program to implement FCFS Disk Scheduling algorithm.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, i;
    int head;
    int request[100];
    int total_movement = 0;
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the disk request sequence:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &request[i]);
    }
    printf("Enter initial head position: ");
    scanf("%d", &head);
    // FCFS Disk Scheduling
    for (i = 0; i < n; i++) {
        total_movement += abs(request[i] - head);
        head = request[i];
    }
    printf("\nTotal head movement = %d cylinders\n", total_movement);
    return 0;
}
```

Q7: Write an OS program to implement SSTF algorithm Disk Scheduling algorithm.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, i, j;
    int head, min, index;
    int request[100], visited[100] = {0};
    int total_movement = 0;
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the disk request sequence:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &request[i]);
    }
    printf("Enter initial head position: ");
```

```
    scanf("%d", &head);
    // SSTF Disk Scheduling
    for (i = 0; i < n; i++) {
        min = 9999;
        for (j = 0; j < n; j++) {
            if (!visited[j]) {
                int distance = abs(request[j] - head);
                if (distance < min) {
                    min = distance;
                    index = j;
                }
            }
        }
        visited[index] = 1;
        total_movement += min;
        head = request[index];
    }
    printf("\nTotal head movement = %d cylinders\n", total_movement);
    return 0;
}
```

Q8: Write an OS program to implement SCAN Disk Scheduling algorithm.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, i, j;
    int head, disk_size;
    int request[100], temp;
    int total_movement = 0;
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the disk request sequence:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &request[i]);
    }
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size: ");
    scanf("%d", &disk_size);
    // Sort the request array
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
```

```
            if (request[i] > request[j]) {
                temp = request[i];
                request[i] = request[j];
                request[j] = temp;
            }
        }
    }
    // Move towards right
    for (i = 0; i < n; i++) {
        if (request[i] >= head) {
            total_movement += abs(request[i] - head);
            head = request[i];
        }
    }
    // Move to end of disk
    total_movement += abs((disk_size - 1) - head);
    head = disk_size - 1;
    // Move towards left
    for (i = n - 1; i >= 0; i--) {
        if (request[i] < head) {
            total_movement += abs(request[i] - head);
            head = request[i];
        }
    }
    printf("\nTotal head movement = %d cylinders\n", total_movement);
    return 0;
}
```

Q9: Write an OS program to implement C-SCAN algorithm Disk Scheduling algorithm.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, i, j;
    int head, disk_size;
    int request[100], temp;
    int total_movement = 0;
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the disk request sequence:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &request[i]);
    }
```

```
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size: ");
    scanf("%d", &disk_size);
    // Sort the request array
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (request[i] > request[j]) {
                temp = request[i];
                request[i] = request[j];
                request[j] = temp;
            }
        }
    }
    // Move towards right
    for (i = 0; i < n; i++) {
        if (request[i] >= head) {
            total_movement += abs(request[i] - head);
            head = request[i];
        }
    }
    // Move to end of disk
    total_movement += abs((disk_size - 1) - head);
    head = disk_size - 1;
    // Jump to beginning of disk
    total_movement += abs(head - 0);
    head = 0;
    // Continue servicing remaining requests
    for (i = 0; i < n; i++) {
        if (request[i] < head) {
            total_movement += abs(request[i] - head);
            head = request[i];
        }
    }
    printf("\nTotal head movement = %d cylinders\n", total_movement);
    return 0;
}
```

Assignment 4 (1 mpicc filename 2 mpiexec –n 4 ./filename)

Q10: Write an MPI program to calculates sum of randomly generated 1000 numbers (stored in array) on a cluster.

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <mpi.h>
#define N 1000
int main(int argc, char *argv[])
{
   int rank, size;
   int i;
   int local_n;
   int local_sum = 0, total_sum = 0;
   int *numbers = NULL;
   int *local_numbers;
   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);
   local_n = N / size;
   local_numbers = (int *)malloc(local_n * sizeof(int));
   if (rank == 0)
   {
      numbers = (int *)malloc(N * sizeof(int));
      srand(123);
      for (i = 0; i < N; i++)
         numbers[i] = rand() % 10;   // random numbers 0–9
   }
   MPI_Scatter(numbers, local_n, MPI_INT,
            local_numbers, local_n, MPI_INT,
            0, MPI_COMM_WORLD);
   for (i = 0; i < local_n; i++)
      local_sum += local_numbers[i];
   MPI_Reduce(&local_sum, &total_sum, 1, MPI_INT,
           MPI_SUM, 0, MPI_COMM_WORLD);
   if (rank == 0)
      printf("Sum of 1000 random numbers = %d\n", total_sum);
   free(local_numbers);
   if (rank == 0)
      free(numbers);
   MPI_Finalize();
   return 0;
}
```

Q11: Write an MPI program to find the min and max number from randomly generated 1000 numbers (stored in array) on a cluster (Hint: Use MPI_Reduce).

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <mpi.h>
#include <limits.h>
#define N 1000
int main(int argc, char *argv[])
{
   int rank, size;
   int i;
   int local_n;
   int local_min = INT_MAX;
   int local_max = INT_MIN;
   int global_min, global_max;
   int *numbers = NULL;
   int *local_numbers;
   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);
   local_n = N / size;
  local_numbers = (int *)malloc(local_n * sizeof(int));
   if (rank == 0)
   {
      numbers = (int *)malloc(N * sizeof(int));
      srand(123);
      for (i = 0; i < N; i++)
         numbers[i] = rand() % 1000;   // random numbers 0–999
   }
   MPI_Scatter(numbers, local_n, MPI_INT,
            local_numbers, local_n, MPI_INT,
            0, MPI_COMM_WORLD);
   for (i = 0; i < local_n; i++)
   {
      if (local_numbers[i] < local_min)
         local_min = local_numbers[i];
      if (local_numbers[i] > local_max)
         local_max = local_numbers[i];
   }
   MPI_Reduce(&local_min, &global_min, 1, MPI_INT,
            MPI_MIN, 0, MPI_COMM_WORLD);
   MPI_Reduce(&local_max, &global_max, 1, MPI_INT,
            MPI_MAX, 0, MPI_COMM_WORLD);
   if (rank == 0)
   {
```

```
        printf("Minimum number = %d\n", global_min);
        printf("Maximum number = %d\n", global_max);
    }
    free(local_numbers);
    if (rank == 0)
        free(numbers);
    MPI_Finalize();
    return 0;
}
```

Q12: Write a review paper on comparative study of different mobile operating system.

```
#include <stdio.h>
int main() {
    printf("=========================================================\n");
    printf("      REVIEW PAPER ON MOBILE OPERATING SYSTEMS      \n");
    printf("=========================================================\n\n");
    printf("1. Android\n");
    printf("---------------------------------------------\n");
    printf("Developer      : Google\n");
    printf("Kernel Type    : Linux Kernel\n");
    printf("Open Source    : Yes (Mostly Open Source)\n");
    printf("Used In        : Samsung, OnePlus, Xiaomi etc.\n");
    printf("Advantages     : Customizable, Large App Store\n");
    printf("Disadvantages  : Fragmentation, Security Risks\n\n");
    printf("2. iOS\n");
    printf("---------------------------------------------\n");
    printf("Developer      : Apple Inc.\n");
    printf("Kernel Type    : XNU Kernel\n");
    printf("Open Source    : No (Closed Source)\n");
    printf("Used In        : iPhone, iPad\n");
    printf("Advantages     : High Security, Optimized Performance\n");
    printf("Disadvantages  : Limited Customization, Expensive Devices\n\n");
    printf("3. Windows Phone\n");
    printf("---------------------------------------------\n");
    printf("Developer      : Microsoft\n");
    printf("Kernel Type    : Windows NT Kernel\n");
    printf("Open Source    : No\n");
    printf("Advantages     : Smooth UI, Office Integration\n");
    printf("Disadvantages  : Limited App Support\n\n");
    printf("4. BlackBerry OS\n");
    printf("---------------------------------------------\n");
    printf("Developer      : BlackBerry Limited\n");
```

```
    printf("Focus          : Enterprise Security\n");
    printf("Advantages     : Strong Security Features\n");
    printf("Disadvantages  : Very Limited App Ecosystem\n\n");
    printf("=========================================================\n");
    printf("Conclusion:\n");
    printf("Android dominates global market due to flexibility.\n");
    printf("iOS is preferred for security and performance.\n");
    printf("Other OS like Windows and BlackBerry declined due to\n");
    printf("poor app ecosystem and market competition.\n");
    printf("=========================================================\n");
    return 0;
}
```